

SELECT Query

```
SELECT col1, col2
FROM table
JOIN table2 ON table1.col = table2.col
WHERE condition
GROUP BY column_name
HAVING condition
ORDER BY col1 ASC|DESC;
```

SELECT Keywords

DISTINCT: Removes duplicate results	SELECT DISTINCT product_name FROM product;
BETWEEN: Matches a value between two other values (inclusive)	SELECT product_name FROM product WHERE price BETWEEN 50 AND 100;
IN: Matches to any of the values in a list	SELECT product_name FROM product WHERE category IN ('Electronics', 'Furniture');
LIKE: Performs wildcard matches using _ or %	SELECT product_name FROM product WHERE product_name LIKE '%Desk%';

Joins

```
SELECT t1.*, t2.*
FROM t1
join_type t2 ON t1.col = t2.col;
```

Table 1	Table 2	
A	A	
B	B	
C	D	
INNER JOIN: show all matching records in both tables.		
A	A	
B	B	
LEFT JOIN: show all records from left table, and any matching records from right table.		
A	A	
B	B	
C		
RIGHT JOIN: show all records from right table, and any matching records from left table.		
A	A	
B	B	
	D	
FULL JOIN: show all records from both tables, whether there is a match or not.		
A	A	
B	B	
C		
	D	

CASE Statement

Simple Case	CASE name WHEN 'John' THEN 'Name John' WHEN 'Steve' THEN 'Name Steve' ELSE 'Unknown' END
Searched Case	CASE WHEN name='John' THEN 'Name John' WHEN name='Steve' THEN 'Name Steve' ELSE 'Unknown' END

Common Table Expression

```
WITH queryname AS (  
SELECT col1, col2  
FROM firsttable)  
SELECT col1,col2..  
FROM queryname...;
```

Modifying Data

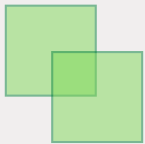
Insert	INSERT INTO tablename (col1, col2...) VALUES (val1, val2);
Insert from a Table	INSERT INTO tablename (col1, col2...) SELECT col1, col2...
Insert Multiple Rows	INSERT INTO tablename (col1, col2...) VALUES (valA1, valB1), (valA2, valB2), (valA3, valB3);
Update	UPDATE tablename SET col1 = val1 WHERE condition;
Update with a Join	UPDATE t SET col1 = val1 FROM tablename t INNER JOIN table x ON t.id = x.tid WHERE condition;
Delete	DELETE FROM tablename WHERE condition;

Indexes

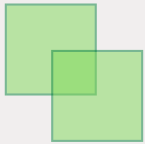
Create Index	CREATE INDEX indexname ON tablename (cols);
Drop Index	DROP INDEX indexname;

Set Operators

UNION: Shows unique rows from two result sets.



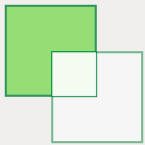
UNION ALL: Shows all rows from two result sets.



INTERSECT: Shows rows that exist in both result sets.



MINUS: Shows rows that exist in the first result set but not the second.



Aggregate Functions

- SUM: Finds a total of the numbers provided
- COUNT: Finds the number of records
- AVG: Finds the average of the numbers provided
- MIN: Finds the lowest of the numbers provided
- MAX: Finds the highest of the numbers provided

Common Functions

- LENGTH(string): Returns the length of the provided string
- INSTR(string, substring): Returns the position of the substring within the specified string.
- CAST(expression AS datatype): Converts an expression into the specified data type.
- ADDDATE(input_date, days): Adds a number of days to a specified date.
- NOW: Returns the current date, including time.
- CEILING(input_val): Returns the smallest integer greater than the provided number.
- FLOOR(input_val): Returns the largest integer less than the provided number.
- ROUND(input_val, [round_to]): Rounds a number to a specified number of decimal places.
- TRUNCATE(input_value, num_decimals): Truncates a number to a number of decimals.
- REPLACE(whole_string, string_to_replace, replacement_string): Replaces one string inside the whole string with another string.
- SUBSTRING(string, start_position): Returns part of a value, based on a position and length.

Create Table

Create Table	CREATE TABLE tablename (column_name data_type);
Create Table with Constraints	CREATE TABLE tablename (column_name data_type NOT NULL, CONSTRAINT pkeyname PRIMARY KEY (col), CONSTRAINT fkname FOREIGN KEY (col) REFERENCES other_table(col_in_other_table), CONSTRAINT ucname UNIQUE (col), CONSTRAINT ckname CHECK (conditions));
Create Temporary Table	CREATE TEMPORARY TABLE tablename (colname datatype);
Drop Table	DROP TABLE tablename;

Alter Table

Add Column	ALTER TABLE tablename ADD columnname datatype;
Drop Column	ALTER TABLE tablename DROP COLUMN columnname;
Modify Column	ALTER TABLE tablename CHANGE columnname newcolumnname newdatatype;
Rename Column	ALTER TABLE tablename CHANGE COLUMN currentname TO newname;
Add Constraint	ALTER TABLE tablename ADD CONSTRAINT constraintname constrainttype (columns);
Drop Constraint	ALTER TABLE tablename DROP constraint_type constraintname;
Rename Table	ALTER TABLE tablename RENAME TO newtablename;

Window/Analytic Functions

```
function_name ( arguments ) OVER (  
[query_partition_clause]  
[ORDER BY order_by_clause  
[windowing_clause] ] )
```

Example using RANK, showing the student details and their rank according to the fees_paid, grouped by gender:

```
SELECT  
student_id, first_name, last_name, gender, fees_paid,  
RANK() OVER (  
PARTITION BY gender ORDER BY fees_paid  
) AS rank_val  
FROM student;
```

Subqueries

Single Row	SELECT id, last_name, salary FROM employee WHERE salary = (SELECT MAX(salary) FROM employee);
Multi Row	SELECT id, last_name, salary FROM employee WHERE salary IN (SELECT salary FROM employee WHERE last_name LIKE 'C%');

Numeric

BIT (n)	Bit-value type. Parameter “n” indicates the number of bits per value, from 1 to 64. Default 1.
TINYINT (n) [UNSIGNED]	A very small integer. Can be signed or unsigned. Signed: -128 to 127, Unsigned: 0 to 255.
SMALLINT (n) [UNSIGNED]	A small integer. Can be signed or unsigned. Signed: -32,768 to 32,767, Unsigned: 0 to 65,535
MEDIUMINT (n) [UNSIGNED]	A medium-sized integer. Can be signed or unsigned. Signed: -8,388,608 to 8,388,607,Unsigned: 0 to 16,777,215
INT (n) [UNSIGNED]	A normal-sized integer. Can be signed or unsigned. Signed: -2,147,483,648 to 2,147,483,647,Unsigned: 0 to 4,294,967,295
INTEGER (n) [UNSIGNED]	A synonym for INT. Can be signed or unsigned. Signed: -2,147,483,648 to 2,147,483,647, Unsigned: 0 to 4,294,967,295
BIGINT (n) [UNSIGNED]	A large integer. Can be signed or unsigned. Signed: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Unsigned: 0 to 18,446,744,073,709,551,615
DECIMAL (n [, d]) [UNSIGNED]	A number with decimal places. Parameter “n” is the precision or number of digits, and “d” is the number of digits after the decimal point (the scale). Maximum for n is 65 and maximum for d is 30. (Note the UNSIGNED parameter is deprecated)
DEC (n [,d])	Synonym for DECIMAL
NUMERIC (n [,d])	Synonym for DECIMAL
FIXED (n [,d])	Synonym for DECIMAL
FLOAT (n [,d])	A small single-precision floating-point number. Uses parameters of “n” for number of digits and “d” for number of digits after the decimal place. Note: the parameters of FLOAT are deprecated as of v8.0.17 Range: -3.402823466E+38 to -1.175494351E-38, 0, and 1.175494351E-38 to 3.402823466E+38
DOUBLE (n [,d])	A normal-sized double-precision floating-point number. Uses parameters of “n” for number of digits and “d” for number of digits after the decimal place. Note: the parameters of FLOAT are deprecated as of v8.0.17 Range: -1.7976931348623157E+308 to -2.2250738585072014E-308, 0, and 2.2250738585072014E-308 to 1.7976931348623157E+308
DOUBLE PRECISION	Synonym for DOUBLE
REAL	Synonym for DOUBLE

Date

DATE	A date value (no time). Range 1000-01-01 to 9999-12-31
DATETIME (fsp)	A date and time value. The parameter “fsp” is fractional seconds precision or the number of fractional seconds that can be stored. Range: 1000-01-01 00:00:00.000000 to 9999-12-31 23:59:59.999999
TIMESTAMP (fsp)	A timestamp value, stores date and time. Has a smaller range than DATETIME. The parameter “fsp” is fractional seconds precision or the number of fractional seconds that can be stored. Range: 1970-01-01 00:00:01.000000 UTC to 2038-01-19 03:14:07.999999
TIME (fsp)	A time value. The parameter “fsp” is fractional seconds precision or the number of fractional seconds that can be stored Range: -838:59:59.000000 to 838:59:59.000000
YEAR	A year in a 4-digit format. Range: 1901 to 2155

Character

CHAR (n)	A fixed-length string. Right-padded with spaces up to the specified length of “n”. Up to 255 bytes.
VARCHAR (n)	A variable-length string. The length parameter of “n” can be from 0 to 65,535. Up to 65,535 bytes.
BINARY (n)	Similar to CHAR but stores binary byte strings rather than nonbinary strings. Parameter “n” is the number of bytes.
VARBINARY (n)	Similar to VARCHAR but stores binary byte strings rather than nonbinary strings. Parameter “n” is the number of bytes.
BLOB (n)	A BLOB column that can store a value up to “n” bytes. Up to 65,535 bytes.
TINYBLOB	A BLOB column with a smaller maximum length, up to 255 bytes.
TEXT (n)	A text column, and parameter “n” is the maximum number of bytes. Up to 65,535 bytes bytes.
TINYTEXT	A text column with a smaller maximum length, up to 255 bytes.
MEDIUMBLOB	A BLOB with a higher maximum length than BLOB. Up to 16,777,215 (2^24 – 1) bytes
MEDIUMTEXT	A text column with a higher maximum length than TEXT. Up to 16,777,215 (2^24 – 1) bytes
LOBLOB	A BLOB column with a high maximum length. Up to 4,294,967,295 or 4GB (2^32 – 1) bytes
LONGTEXT	A text column with a high maximum length. Up to 4,294,967,295 or 4GB (2^32 – 1) bytes
ENUM (value_list)	A string object that can have only one value from the list of values specified, or NULL. Can have up to 65,535 items in its list.
SET (value_list)	A string object that can have zero or more values from the list of values specified. Can have up to 64 items in its list.
JSON	Stores JSON Data

Other

GEOMETRY	Stores geometry values of any type
POINT	Stores a point in geometry
LINESTRING	Stores a line shape
POLYGON	Stores a polygon shape
MULTIPOINT	Stores a collection of points
MULTILINESTRING	Stores a collection of lines
MULTIPOLYGON	Stores a collection of polygons
GEOMETRYCOLLECTION	Store a collection of geometry objects
BOOL	Synonym for TINYINT(1). Zero is false, nonzero values are true.
BOOLEAN	Synonym for TINYINT(1). Zero is false, nonzero values are true.